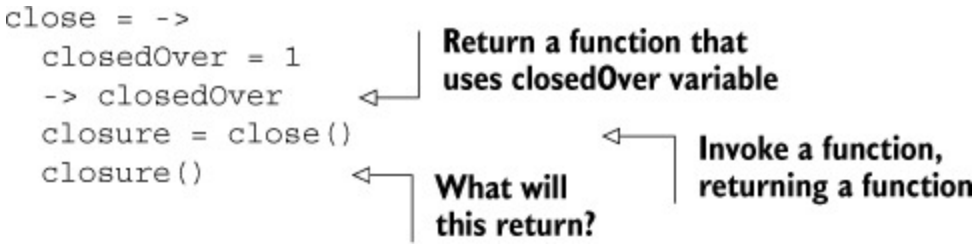
Although you have access to the arguments passed to a containing function, if you use the same argument name for a function scoped inside that containing function, then the inner function won't have access to the outer function argument. The name of an argument will shadow the name of a variable or argument in an outer scope.

Functions as return values

Perhaps you're thinking at this point that as awesome as this all sounds, nested scopes don't seem to buy you much. Why would you want to nest functions like this? You're about to find out.

You see, there's one more thing you can do with functions as values: you can return a function from a function. Pause for a minute and think what might happen to the scoped variables when a function is returned from a function:



This is your introduction to closures.

3.6. Closures

To newcomers, the idea of *closures* is often shrouded in mystery, so it's useful to first look at where they come from. Peter Landin coined the term *closure*, with the meaning used in CoffeeScript, in 1964 when he was implementing a virtual machine called the SECD[4] machine. Strictly speaking, it means that functions *close over* the free variables in their lexical scope. This has some profound importance that you need to see in action to appreciate. In this section you'll learn how closures work and how they not only help you get away from problematic global state but also